

AI Security & Responsible AI Cheat Sheet

Baseline written 2026-10-01: OWASP Top 10 for LLM Applications 2026 (Aug 2026), OWASP Top 10 for Agentic Applications 2026 (2025-12-09), MITRE ATLAS, NIST AI RMF 1.0 + AI 600-1 (Jul 2024), EU AI Act (Reg. 2024/1689) + Digital Omnibus on AI (unconfirmed on EUR-Lex), garak 0.17.0, PyRIT 1.1.0, promptfoo 0.123.1, NeMo Guardrails 0.24.1, Guardrails AI 0.11.0, Presidio 2.2.364, safetensors 0.8.0, model-signing 1.1.1, LangChain 1.4.3 (Python 3.12), Spring AI 2.0.1 (Spring Boot 4.1.x, Java 21). Snippets are illustrative, not compiled; no scan results or benchmark numbers quoted. Not legal advice.

TRUST BOUNDARIES

- Boundaries: user → app; app → model; retrieval store → prompt; model → tools; tools → backends; model output → renderer/interpreter; supply chain (models, packages, MCP servers) → app.
- Everything the model reads (docs, web pages, tool results, email, memory) is **data from an untrusted author**, not instructions.
- Everything the model writes is **untrusted input** to the next component.
- Threat-model flow: draw architecture → mark boundaries → STRIDE per crossing → map to OWASP / ATLAS → pick controls → test → revisit on change. MAESTRO adds agent layers.
- Key ATLAS: T0051 prompt injection, T0054 jailbreak, T0020 poison data, T0010 supply chain, T0024 exfiltration via API, T0029 DoS, T0056 extract system prompt, T0057 data leakage, T0070 RAG poisoning, T0053 agent tool invocation.

OWASP LLM 2026 ← 2025 / 2023

2026	2025	2023
LLM01 Prompt Injection	01	01
LLM02 Sensitive Info Disclosure	02	06
LLM03 Excessive Agency	06	08 (+07)
LLM04 Supply Chain	03	05
LLM05 Data & Model Poisoning	04	03
LLM06 Unbounded Consumption	10	04 (+10)
LLM07 Misinformation	09	09
LLM08 Hidden Context Exposure	07 System Prompt Leakage	–
LLM09 Vector & Embedding Weak.	08	–
LLM10 Improper Output Handling	05	02

Book editions (v1.1, 2023) are outdated: use the 2026 list.

OWASP AGENTIC (ASI)

ASI01 Agent Goal Hijack	ASI06 Memory Poisoning
ASI02 Tool Misuse	ASI07 Inter-Agent Communication
ASI03 Identity Abuse	ASI08 Cascading Failures
ASI04 Supply Chain	ASI09 Trust Exploitation
ASI05 Code Execution	ASI10 Rogue Agents

Short names taken from secondary summaries; re-check the official PDF. Agent risk = LLM risk × autonomy, memory, identity and tools: give each agent its own identity, scoped credentials and a kill switch.

PROMPT-INJECTION DEFENCE LAYERS

- No complete fix: assume injection succeeds and limit the blast radius.
- 1 Input:** length caps, normalisation, injection classifier (Prompt Guard 2 / Llama Guard 4).
- 2 Prompt:** separate roles; delimit and label retrieved text as data; keep secrets out of prompts.
- 3 Retrieval:** filter by user ACL; sanitise ingested documents.
- 4 Tools:** least privilege, typed args, confirmation (see below).
- 5 Output:** schema validation, encoding, link/URL allow-list, no auto-rendered remote images (exfiltration).
- 6 Monitor:** log, alert, red-team in CI.
- Python: LangChain middleware; Java: Spring AI advisors (SafeGuardAdvisor).

TOOL / MCP LEAST-PRIVILEGE RULES

- Narrow typed tools, never raw shell or SQL; allow-list per agent and per task.
- Validate every argument server-side; model output never authorises.
- Act as the **end user** (delegated token, per-user authorisation), not as a service super-user.
- Human approval for destructive/irreversible/external actions (HumanInTheLoopMiddleware).
- Sandbox code execution: no network, read-only FS, CPU/time limits.
- Step, tool-call and model-call budgets (ToolCallLimitMiddleware, ModelCallLimitMiddleware).
- MCP: pin and review servers like dependencies, scope OAuth tokens, treat tool descriptions as untrusted, no token passthrough.

OUTPUT-HANDLING RULES

- Treat model text as untrusted input to the sink.
- Constrain shape: schema-validated structured output; reject on failure.
- Encode for the sink: HTML-escape (XSS in chat UIs), safe Markdown, parameterised SQL, argument arrays for commands.
- Never build queries, commands, URLs or file paths from model text.
- Strip/allow-list links and images; safe rendering per channel.
- Validate in the agent loop (Python) and the web layer/advisors (Java).

SENSITIVE INFO & HIDDEN CONTEXT

- Secrets never in prompts, tool schemas or memory: the system prompt **will** leak (LLM08).
- Redact PII before the model and in logs (Presidio, PIIMiddleware).
- Do not fine-tune on data users may not see; per-tenant memory; retention limits.

RAG / VECTOR CHECKLIST

- Document-level ACL **in the retrieval filter**, not after generation.
- Tenant isolation: collection/namespace or enforced filter per tenant.
- Embeddings are data: can be inverted, so protect and encrypt like source.
- Ingestion: provenance, signing, sanitisation, quarantine for new sources.
- Query time: top-k limits, citation of sources, refuse low-relevance answers.
- Test cross-tenant leaks and poisoned-document injection (ATLAS T0070).

SUPPLY-CHAIN CONTROLS

- Prefer **safetensors**; refuse pickle (use_safetensors=True, trust_remote_code=False).
- Pin models/datasets to immutable commit SHA.
- Scan, but do not depend on scanning.
- Inventory: CycloneDX ML-BOM / SBOM.
- Verify signatures (Sigstore model-signing).
- Pin Maven/PyPI dependencies, verify hashes.
- Package names suggested by models are untrusted (slopsquatting).
- Review plugins, skills, MCP servers as dependencies.

CONSUMPTION LIMITS (LLM06)

- Per-key rate, input/output token and context caps, timeouts, concurrency limits, step/recursion limits for agents.
- Worst-case bound per key and window:

C_max = R · (n_in^max · p_in + n_out^max · p_out)
e.g. 1000 · (8000 · 3e-6 + 500 · 15e-6) = 31.5 units/day

Budgets and alerts per tenant; cache; fail closed on budget exhaustion.

MISINFORMATION & HALLUCINATION

- Ground answers in retrieved sources and cite them; abstain when evidence is thin.
- Verify high-stakes output (legal, medical, code, packages); show uncertainty and AI disclosure.
- Measure faithfulness; keep a human in the loop for consequential decisions.

GUARDRAIL OPTIONS

Tool	Role
Llama Guard 4 (12B)	content-safety classifier, in/out
Llama Prompt Guard 2 (86M)	injection / jailbreak detector
ShieldGemma	safety classifier
NeMo Guardrails 0.24.1	programmable dialogue rails
Guardrails AI 0.11.0	validators, structured output
Presidio 2.2.364	PII detect/anonymise
Provider moderation APIs	hosted category filters
Spring AI	SafeGuardAdvisor, moderation model

Layers: input / retrieval / output / action. Budget latency; decide fail-open vs **fail-closed** per layer.

RED-TEAMING COMMANDS

```
python -m garak --list_probes
python -m garak --model_type openai \
  --model_name gpt-4o-mini --probes promptinject
python -m garak --model_type rest -G cfg.json \
  --probes promptinject --generations 5
npx promptfoo@latest redteam init
npx promptfoo@latest redteam run
```

PyRIT 1.1.0: scripted multi-turn attacks with scorers. Test the whole app endpoint, not only the model. Results are statistical: report rates, rerun after every change, gate CI. Commands here are from official docs only, not run.

LLMSECOPS

- Security at every lifecycle stage: data, build, eval, deploy, operate.
- Audit: scope, threat model, controls, red team, remediate, re-test.
- Redact prompts/responses in logs; OTel GenAI + structured security events to the SIEM.
- AI incident runbook: contain (kill switch, rotate keys), investigate, notify, learn.
- Secrets in a vault; short-lived keys.

AI-ASSISTED DEVELOPMENT

- Review AI-generated code like any untrusted PR; run SAST and secret scans (detect-secrets).
- Scope agent permissions and repo access; protect config and rules files from tampering.
- Vet MCP servers and skills; no secrets in prompts or context.

NIST FUNCTION MAP

Govern	owner, risk appetite, approved models, incident process
Map	purpose, users, data inventory, threat model, misuse
Measure	evals, red-team reports, monitoring
Manage	prioritise, treat, respond, communicate

AI 600-1 (Jul 2024) 12 risks: CBRN; confabulation; dangerous/violent/hateful content; data privacy; environmental impacts; harmful bias and homogenization; human-AI configuration; information integrity; information security; intellectual property; obscene/degrading/abusive content; value chain and component integration.

EU AI ACT TIMELINE (NOT LEGAL ADVICE)

2025-02-02	prohibitions + AI literacy
2025-08-02	GPAl obligations, governance
Aug 2026	Art. 50 transparency (Commission page)
2026-12-02	Art. 50(2) marking, existing systems (unconfirmed)
Dec 2026	new prohibition: non-consensual intimate content
2027-12-02	Annex III high-risk (Omnibus, unconfirmed)
2028-08-02	Annex I high-risk (Omnibus, unconfirmed)

Tiers: prohibited, high-risk, transparency, minimal. Know your role: provider vs deployer. Digital Omnibus on AI reportedly Reg. (EU) 2026/1744: verify on EUR-Lex.

RAISE / RESPONSIBLE-AI CHECKLIST

- RAISE (paraphrased): narrow the domain; treat data/models as supply chain; keep humans able to intervene; test adversarially; monitor after release.
- ☐ Purpose, users, out-of-scope uses written down
- ☐ AI disclosure and source citations visible
- ☐ Fairness checks per group and language, tolerated gap set
- ☐ Approval gates and escalation tested
- ☐ Accessibility tested with assistive tech
- ☐ Security controls and red-team results current
- ☐ NIST mapping and EU tier assessment current
- ☐ Incident and rollback procedure exists